

Project Pipeline Components for Model Construction

AMD Options Pipeline: CRR Binomial Pricing + Kelly Criterion Sizing

github.com/dantzlerdc/CS495-Deep-Scholar

Project Overview

This project implements a six-stage quantitative pipeline for AMD equity options. It prices American-style call and put options using the Cox-Ross-Rubinstein (CRR) binomial model (no dividends, N=100 steps), computes numerical Greeks, measures the signed mispricing edge between the model price and the Fidelity limit price, and sizes each trade using the Kelly Criterion (full, half, and quarter variants). A Monte Carlo simulation evaluates the P&L distribution of each Kelly strategy across 1,000 hypothetical trades. All outputs are saved as CSV files and PNG plots to project/outputs/.

Pipeline Stages at a Glance

#	File	What it does
1	data.py	Ingest AMD chain; fetch realized volatility from Yahoo Finance
2	tree.py	Build CRR lattice; price all four tickets with early exercise
3	greeks.py	Compute Delta, Gamma, Theta, Vega, Rho via finite differences
4	edge.py	Signed mispricing edge = (V_model - V_market) / V_market
5a	kelly.py	Full / half / quarter-Kelly fractions and dollar positions
5b	simulation.py	Monte Carlo P&L: Sharpe, max drawdown, hit rate (1,000 trades)
6	main.py	Orchestrates all stages; generates 4 plots to outputs/

Quick Start

```
make setup # create .venv and install all dependencies
make run   # run the full 6-stage pipeline
```

config.yaml

Pipeline configuration -- all AMD option chain inputs in one place.

Purpose

Stores every parameter the pipeline needs: the AMD stock price ($S = \$341.35$), risk-free rate ($r = 5.3\%$), collection and expiry dates, the CRR step count ($N = 100$), and the four Fidelity trade tickets (strike, action, option type, market limit price, implied volatility, break-even). Editing this file is the only change needed to re-run the pipeline for a different ticker, expiry, or set of strikes.

Key Functions / Items

amd.S / amd.r / dates	-- spot price, risk-free rate, collection and expiry dates
model.N	-- number of binomial tree steps (default 100)
tickets[]	-- list of four AMD option tickets with K, V_market, iv, break_even
kelly.capital / kelly.min_edge	-- base capital and minimum edge threshold

How to Run

Not executed directly. Read automatically by data.py via load_config().
Open with any text editor to update inputs before running the pipeline.

data.py***Stage 1 -- Data ingestion and preprocessing.*****Purpose**

Loads config.yaml, computes time-to-expiry T in years, downloads six months of AMD daily closing prices from Yahoo Finance via yfinance, and calculates 30-day and 60-day annualized realized volatility. Assembles all inputs (S , K , r , T , IV , V_{market} , moneyness, IV -premium over realized vol) into a single pandas DataFrame that every downstream stage consumes.

Key Functions / Items

load_config(path)	-- parses config.yaml into a Python dict
compute_time_to_expiry(d0, d1)	-- returns T in years between two date strings
fetch_realized_volatility(ticker)	-- downloads history, returns rv_{30} and rv_{60}
build_option_chain_df(cfg, T , rv)	-- returns the master input DataFrame

How to Run

Imported as a module by main.py -- not run standalone.

To test in isolation (run from the project/ directory):

```
python3 -c "from data import *; cfg=load_config();
T=compute_time_to_expiry(cfg['amd']['collection_date'],
cfg['amd']['expiry_date']); rv=fetch_realized_volatility('AMD');
print(T, rv)"
```

tree.py**Stage 2 -- CRR American binomial tree pricer (core model).****Purpose**

Implements the Cox-Ross-Rubinstein (1979) binomial lattice for American-style options using vectorized NumPy. At each of N steps the stock moves up by $u = \exp(\sigma \cdot \sqrt{dt})$ or down by $d = 1/u$. Risk-neutral probability is $p = (\exp(r \cdot dt) - d) / (u - d)$ with no dividend adjustment (per project scope). Backward induction enforces American early exercise at every node by taking $\max(\text{intrinsic}, \text{continuation})$. Also returns the early-exercise boundary S^* for put options at each time step.

Key Functions / Items

```
price_american_option(S, K, r, sigma, T, N, option_type) -- main entry point
Returns (price: float, exercise_boundary: ndarray of length N+1)
option_type accepts 'call' or 'put'
exercise_boundary[i] = highest S at step i where early exercise beats waiting (puts)
```

How to Run

Imported by `greeks.py`, `edge.py`, and `main.py`.
To price one option directly (run from the project/ directory):

```
python3 -c "from tree import price_american_option;
p, b = price_american_option(341.35, 350, 0.053, 0.7392,
18/365, 100, 'call'); print(p)"
```

greeks.py

*Stage 3 -- Numerical Greeks via centered finite differences.***Purpose**

Computes all six Greeks for each AMD option ticket by re-running the CRR tree with small input perturbations. Delta and Gamma perturb spot price by 1% of S; Theta perturbs time by one calendar day; Vega and Rho perturb implied volatility and the risk-free rate by 1 percentage point each. Results are validated against the Greeks displayed on the Fidelity option chain.

Key Functions / Items

```
compute_greeks(S, K, r, sigma, T, N, option_type) -- returns dict of all six Greeks
Delta = (V(S+dS) - V(S-dS)) / (2*dS)
Gamma = (V(S+dS) - 2*V(S) + V(S-dS)) / dS^2
Theta = (V(T-1day) - V(T)) / (1/365) [per calendar day]
Vega = (V(sigma+0.01) - V(sigma-0.01)) / 0.02 [per vol point]
Rho = (V(r+0.01) - V(r-0.01)) / 0.02 [per 100 bps]
```

How to Run

Imported by main.py. Results printed to console and saved to outputs/greeks.csv.
To compute Greeks for one ticket (run from the project/ directory):

```
python3 -c "from greeks import compute_greeks;
g = compute_greeks(341.35, 350, 0.053, 0.7392, 18/365, 100, 'call');
print(g)"
```

edge.py**Stage 4 -- Mispricing edge computation.****Purpose**

For each ticket, prices the option with the CRR tree and computes the signed mispricing edge: $\text{edge} = (V_{\text{model}} - V_{\text{market}}) / V_{\text{market}}$. A positive edge means the model price exceeds the market price (market is cheap -- favour buying). A negative edge means the market is more expensive than the model says (overpriced -- favour selling). This is the core signal that feeds into Kelly sizing.

Key Functions / Items

```
compute_edges(chain_df, N)  -- iterates over all tickets and returns edge DataFrame
Output columns: ticket_id, V_model, V_market, edge, edge_pct, abs_edge_pct
Positive edge_pct  ->  market underpricing  (buy signal)
Negative edge_pct  ->  market overpricing   (sell signal)
```

How to Run

Imported by main.py. Results printed to console and saved to outputs/edges.csv.
To run in isolation (run from the project/ directory):

```
python3 -c "from data import *; from edge import compute_edges;
cfg=load_config();
T=compute_time_to_expiry(cfg['amd']['collection_date'],
cfg['amd']['expiry_date']);
rv=fetch_realized_volatility('AMD');
df=build_option_chain_df(cfg,T,rv); print(compute_edges(df,100))"
```

kelly.py**Stage 5a -- Kelly Criterion trade sizing.****Purpose**

Converts each ticket's mispricing edge into a concrete position size using the Kelly Criterion: $f^* = (p \cdot b - q) / b$, where p is the model-implied win probability, $b = 1$ (1:1 simplified odds), and $q = 1 - p$. The win probability is derived by shifting a 0.50 baseline by half the signed edge. Tickets with $|edge|$ below min_edge (default 2%) are flagged NO TRADE to filter model noise. Full, half, and quarter-Kelly dollar positions are reported against a configurable capital base (default \$100,000).

Key Functions / Items

```
_edge_to_win_prob(edge, action)          -- maps edge to p, flips sign for sell tickets
kelly_fractions(p, b=1.0)                -- returns f_full, f_half, f_quarter
compute_kelly_recommendations(edge_df, chain_df,
                               capital, min_edge)  -- main entry
Output: dollar_full, dollar_half, dollar_quarter, and trade_signal per ticket
```

How to Run

Imported by main.py. Results saved to outputs/kelly.csv after each run.

simulation.py**Stage 5b -- Monte Carlo P&L simulation.****Purpose**

Simulates 1,000 hypothetical repeat trades under each Kelly variant (full, half, quarter) using the model-implied win probability. Each trade is a binary outcome: win (+ $f \times \text{capital}$) or lose (- $f \times \text{capital}$). Reports cumulative P&L, hit rate, annualized Sharpe ratio, and maximum drawdown. Results illustrate the risk-return trade-off between full-Kelly (maximum long-run growth, higher variance) and fractional-Kelly (lower growth, more stable equity curve) strategies.

Key Functions / Items

```
simulate_pnl(p_win, f, capital, n_trades=1000) -- single-ticket simulation
run_kelly_simulations(kelly_df, capital)      -- all ticket x variant combinations
Output: hit_rate, total_pnl, max_drawdown, sharpe_annualized per variant
Fixed random seed (42) ensures reproducible results
```

How to Run

Imported by main.py. Results saved to outputs/simulation.csv after each run.

main.py

Pipeline runner -- orchestrates all 6 stages end-to-end.

Purpose

The single entry point for the entire project. Calls each stage in sequence: (1) data ingestion, (2) CRR pricing, (3) Greeks, (4) mispricing edge, (5) Kelly sizing and Monte Carlo simulation, (6) plot generation. Prints a formatted report to the console for each stage and writes all results to project/outputs/ as CSV files and PNG plots.

Key Functions / Items

```
run(config_path)    -- top-level function; executes the full pipeline
stage1_ingest()     -- loads config, fetches RV, builds chain DataFrame
stage2_price()      -- CRR prices all four tickets, reports % error
stage3_greeks()     -- computes and prints all Greeks; saves greeks.csv
stage4_edge()       -- computes mispricing edge; saves edges.csv
stage5_kelly()      -- Kelly sizing + Monte Carlo; saves kelly.csv & simulation.csv
Plots: model_vs_market.png, convergence.png, kelly_fractions.png,
      early_exercise_boundary.png (all written to project/outputs/)
```

How to Run

From the repo root with the virtual environment active:

```
make run
```

Or directly from the project/ directory:

```
python3 main.py config.yaml
```

First-time setup (creates .venv and installs all dependencies):

```
make setup
```